

Intro POO

POO

POO, pour **P**rogrammation **o**rientée **O**bjets. Vous rencontrerez aussi parfois son sigle anglophone, **OOP**, pour ***O**bject-**o**riented **P**rogramming*.

La programmation orientée objet est un **paradigme** de programmation dont le principe est de penser un projet informatique comme un ensemble d'unités applicatives autonomes interagissant les unes avec les autres : des objets.

Une application pensée selon les principes de la POO se constitue donc d'un ensemble d'objets fonctionnant de concert pour parvenir à un ensemble de fonctionnalités.

Qu'est-ce qu'un objet ?

Dans son sens large, un objet est la représentation conceptuelle de tout élément matériel de notre réalité : un chat est un objet, un violon est un objet, une planète est un objet. Vous êtes un objet.

Un objet a des propriétés : une voiture a une certaine couleur, une certaine marque, une certaine masse.

Un objet exécute des actions : une voiture peut avancer, reculer, freiner.

Un objet peut se composer d'objets : l'objet voiture, avec ses propriétés et ses actions, est composé d'un objet moteur, d'un objet boîte de vitesses, d'un objet batterie.

Un objet peut interagir avec d'autres objets : l'objet voiture se déplace sur l'objet route, fait le plein de l'objet carburant à l'objet pompe de l'objet station-service.

Simple. On pense le monde comme un immense tout où interagissent des objets. Chaque objet a des caractéristiques qui peuvent évoluer dans le temps ou non : on les appelle des **propriétés**). Chaque objet peut exécuter des actions : on les appelle des **méthodes**).

Quel rapport avec la programmation ?

À travers la programmation orientée objet, on pense un projet informatique comme on vient de penser le monde : des objets qui interagissent avec des objets pour former un tout.

Votre schéma **MVC** requiert un système *routing* : celui-ci sera assuré par un objet **Router**. Votre **Router** appelle des contrôleurs : ceux-ci seront des objets **Controller**. Votre **Controller** manipule des modèles : des objets **Article**, **User**, **Comment**, etc.

Résumé

En programmation orientée objet, tout notre code est pensé comme un agencement d'objets interagissant les uns avec les autres. Fini le code procédural, à présent tout est objet.

Les classes

Classes

Chaque objet est en réalité une **instance** d'une **classe**.

Voyez la **classe** comme la définition abstraite d'un certain type d'objet, et l'**instance** comme une occurrence concrète de cette définition.

Si nous vous disons « Fermez les yeux et pensez à une voiture. » Que voyez-vous ? Juste « une voiture » ? Non, vous ne pensez pas de façon abstraite au concept de voiture. Vous voyez concrètement à une voiture précise : le Scenic gris de papa, la Kia jaune fluo de votre pote Magali en fac' de socio'. Vous voyez une occurrence concrète du concept de voiture, une **implémentation**, parce qu'il vous est pour ainsi dire impossible de visualiser un concept abstrait.

C'est la distinction entre une classe et une instance. « Voiture », c'est une classe. La Yaris noire toute rayée en bas de chez vous, c'est une instance de la classe « Voiture », avec ses données propres.

Propriétés

```
class Movie
{
    public int $year;

    public string $title;
}
```

La classe **Movie** ne représente pas un film spécifique. Elle représente la définition abstraite de ce qu'est un film. Pour manipuler un film en particulier, nous devons créer une instance de cette classe :

```
$mov = new Movie();
$mov->year = 1998;
$mov->title = 'Ringu';
```

À travers le mot-clé **new** suivi du nom de la classe et de parenthèses, nous créons une nouvelle instance de la classe **Movie**. Ensuite nous spécifions des valeurs précises dans ses propriétés jusqu'ici vierges : on dit qu'on **hydrate** l'objet.

Notre variable `$mov` contient désormais une instance de la classe `Movie` modélisant un film très précis.

Il nous est désormais très simple d'afficher, par exemple, le titre de ce film :

```
echo $mov->title; // 'Ringu'
```

Méthodes

Un objet ne se compose pas forcément exclusivement de propriétés. Il peut aussi effectuer des actions. Reprenons le cas de `Movie` très simple :

```
class Movie
{
    public int $year;
    public string $title;

    // Méthode pour afficher une description du film
    public function getDescription(): string
    {
        return "Le film '{$this->title}' a été sorti en {$this->year}.";
    }
}
```

```
// Exemple d'utilisation
$movie = new Movie();
$movie->title = 'Oppenheimer';
$movie->year = 2023;
echo $movie->getDescription(); // Affichera: "Le film
'Oppenheimer' a été sorti en 2023."
```

Ce qu'il faut retenir ici, c'est la syntaxe d'une méthode :

```
public function getDescription(): void
```

Ça ressemble fortement à une fonction, n'est-ce pas ?

C'est une fonction. « Méthode » est le nom que l'on donne à une fonction liée à une classe.

Mais ces histoires de `public` que l'on voit depuis le début ? C'est la **visibilité**. Nous y venons au chapitre suivant sur la notion d'encapsulation.

__construct()

Nous allons aborder rapidement la méthode `__construct`. Il s'agit de ce que PHP appelle une **méthode magique**. Nous n'entrerons pas dans le détail de

toutes les méthodes magiques, la documentation de PHP est là pour cela. Nous vous montrons pour l'instant la plus utilisée :

```
class User
{
    public string name;

    public function __construct()
    {
        echo 'Salut';
    }
}
```

```
$user = new User();
```

Ce code affiche le message **Salut**. Pourquoi ? Parce que la méthode **__construct** est une méthode qui s'exécute automatiquement au moment de l'instanciation de la classe. Quand vous tapez **new User()**, PHP exécute la méthode **__construct** de la classe **User** s'il en existe une.

Nous pouvons aussi passer des paramètres à cette méthode.

```
class User
{
    public string name;

    public function __construct(string $message)
    {
        echo $message;
    }
}
```

```
$user = new User('Salut');
```

Nous pouvons définir des paramètres à la méthode **__construct**. Ces arguments sont ensuite passés entre les parenthèses qui suivent le nom de la classe à l'instanciation.

Le **__construct**, entre autres usages, est très souvent employé pour hydrater une classe dès sa création.

```
class User
{
    public string $name;
    public int $age;

    public function __construct(string $name, int $age)
    {
        $this->name = $name;
        $this->age = $age;
    }
}
```

```
}
```

```
$user = new User('Magali', 22);  
echo $user->age; // 22
```

Vous découvrez ici la variable `$this`. C'est très simple : elle est automatiquement déclarée par PHP et est accessible dans l'ensemble des méthodes d'une classe. Que contient-elle ? L'instance courante. La variable `$this` se réfère toujours à l'instance actuellement manipulée.

Et on utilise ici ce `$this` pour modifier dynamiquement les propriétés de l'instance. On passe deux valeurs au `__construct`, et on les assigne dans la foulée aux propriétés auxquelles on veut les assigner.

Hydrater un objet dans la foulée de son instanciation est l'utilisation la plus commune du `__construct`, mais vous verrez avec le temps que ce n'est pas la seule.

Résumé

À retenir :

- Une **classe** est la définition abstraite de futurs objets ;
- Un objet, ou **instance**, est un exemplaire concret d'une classe ;
- Les objets peuvent avoir des caractéristiques sous la forme de variables liées à une classe : des **propriétés** ;
- Les objets peuvent effectuer des actions sous la forme de fonctions liées à une classe : des **méthodes** ;
- La méthode magique `__construct`, si on la définit, est automatiquement exécutée à la création d'une instance ;
- On peut se référer à une instance depuis l'intérieur de ses méthodes grâce à la variable `$this`.

La visibilité

En PHP, la visibilité est un concept central qui permet de contrôler **l'accès aux propriétés et aux méthodes** d'une classe. Elle constitue le fondement de l'encapsulation, l'un des principes majeurs de la programmation orientée objet (POO). PHP propose trois niveaux de visibilité : **public**, **protected** et **private**.

Les niveaux de visibilité

1. Public

Une propriété ou une méthode déclarée **public** est accessible partout : aussi bien à l'intérieur de la classe qu'en dehors, par des objets extérieurs. Cela en fait le niveau de visibilité le plus permissif.

```
class Utilisateur
{
    public string $nom;

    public function __construct(string $nom)
    {
        $this->nom = $nom;
    }

    public function saluer(): string
    {
        return "Bonjour, " . $this->nom;
    }
}

$utilisateur = new Utilisateur("Alice");
echo $utilisateur->nom; // "Alice"
echo $utilisateur->saluer(); // "Bonjour, Alice"
```

Ici, la propriété **\$nom** et la méthode **saluer** sont ouvertes à tous les usages, sans aucune restriction.

2. Protected

Les éléments marqués `protected` ne sont accessibles qu'à l'intérieur de la classe où ils sont définis, ainsi que dans les classes qui en héritent. Ils permettent donc un certain niveau de contrôle tout en autorisant une extensibilité dans des classes dérivées.

```
class Compte
{
    protected float $solde;

    public function __construct(float $soldeInitial)
    {
        $this->solde = $soldeInitial;
    }

    protected function afficherSolde(): string
    {
        return "Solde : " . $this->solde . "€";
    }
}

class CompteEpargne extends Compte
{
    public function consulterSolde(): string
    {
        return $this->afficherSolde();
    }
}

$epargne = new CompteEpargne(1000);
echo $epargne->consulterSolde(); // "Solde : 1000€"
// echo $epargne->solde; // Erreur : inaccessible
```


3. Private

Le niveau `private` est le plus restrictif. Les propriétés et méthodes privées sont strictement limitées à la classe qui les déclare. Cela permet de protéger totalement les données contre tout accès ou modification externe.

```
class CarteBancaire
{
    private string $codePIN;

    public function __construct(string $codePIN)
    {
        $this->codePIN = $codePIN;
    }

    private function verifierPIN(string $code): bool
    {
        return $this->codePIN === $code;
    }

    public function payer(string $code): string
    {
        if ($this->verifierPIN($code)) {
            return "Païement accepté.";
        }

        return "Païement refusé : code PIN incorrect.";
    }
}

$carte = new CarteBancaire("1234");
echo $carte->payer("1234"); // "Païement accepté."
// echo $carte->codePIN; // Erreur : inaccessible
```

Pourquoi définir une visibilité ?

La visibilité protège les données internes d'une classe en empêchant des manipulations incontrôlées, tout en clarifiant ce qui relève de l'interface publique ou de l'implémentation interne. En limitant l'accès aux parties sensibles, elle permet aussi de faire évoluer le code sans risquer de perturber les dépendances externes.

Les bonnes pratiques

Déclarez vos propriétés privées par défaut et ouvrez leur accès uniquement si nécessaire. Respectez le principe de moindre privilège en exposant uniquement ce qui doit l'être, et explicitiez toujours la visibilité pour renforcer la clarté du code.

Les getters et setters

Les getters et setters sont des méthodes permettant de lire ou de modifier des propriétés privées ou protégées tout en offrant un contrôle sur les valeurs. Par exemple, un setter peut valider une donnée avant de l'affecter à une propriété :

```
class Utilisateur
{
    private string $nom;

    public function getNom(): string
    {
        return $this->nom;
    }

    public function setNom(string $nom): void
    {
        if (empty($nom)) {
            throw new InvalidArgumentException("Le nom ne peut pas être vide.");
        }
        $this->nom = $nom;
    }
}

$user = new Utilisateur();
$user->setNom("Alice");
echo $user->getNom(); // "Alice"
```

Quand les utiliser ou ne pas les utiliser ?

Utilisez des getters et setters lorsque vous devez appliquer des validations, des transformations ou des contrôles supplémentaires sur les données. Ils sont utiles pour protéger l'accès aux propriétés et pour garantir la cohérence des objets. Les getters sont également utiles si vous souhaitez exposer des données "virtuelles" (par exemple le résultat d'un calcul), ou bien si vous pensez que l'implémentation interne risque d'évoluer et que vous voulez vous assurer que l'API publique reste la même.

Cependant, si une propriété n'a pas besoin de validation ou de traitement particulier, leur utilisation peut alourdir le code sans réelle valeur ajoutée. Dans ce cas, privilégiez des propriétés `public` ou `readonly` (PHP 8.1) pour la simplicité.

Static

En programmation orientée objet (POO) en PHP, le mot-clé `static` est utilisé pour définir des propriétés ou des méthodes qui appartiennent à la classe elle-même plutôt qu'à une instance particulière de la classe. Cela signifie que vous pouvez accéder à des éléments statiques sans avoir à instancier la classe, et ils sont partagés entre toutes les instances de la classe.

Les propriétés statiques

Une propriété statique est une variable qui appartient à la classe elle-même plutôt qu'à une instance de la classe. Elle est définie avec le mot-clé `static` et peut être utilisée pour stocker des données partagées entre toutes les instances de la classe. Pour accéder à une propriété statique, vous utilisez le nom de la classe suivi de `::` (opérateur de résolution de portée) et le nom de la propriété.

```
class Exemple {  
    public static $proprieteStatique = "Ceci est une propriété  
    statique."  
}
```

```
// Accéder à une propriété statique  
echo Exemple::$proprieteStatique;
```

Les méthodes statiques

Une méthode statique est une fonction qui appartient à la classe elle-même plutôt qu'à une instance de la classe. Elle est définie avec le mot-clé `static`. Les méthodes statiques peuvent être appelées sans instancier la classe et sont généralement utilisées pour effectuer des opérations qui ne dépendent pas de l'état d'une instance particulière.

```
class Exemple {  
    public static function methodeStatique() {  
        return "Ceci est une méthode statique.";  
    }  
}
```

```
// Appeler une méthode statique  
echo Exemple::methodeStatique();
```

L'utilisation de propriétés et de méthodes statiques est utile lorsque vous avez besoin de partager des données ou d'accéder à des fonctionnalités spécifiques à la classe plutôt qu'à une instance individuelle. Cependant, il est important de noter que l'utilisation abusive de propriétés et de méthodes statiques peut rendre votre code moins souple et plus difficile à tester, car elles introduisent un couplage fort entre **les classes** et peuvent rendre le code moins orienté objet. Utilisez-les judicieusement en fonction des besoins de votre application.

Exercice - Avatar

Le but de cet exercice est de créer un avatar automatique pour un utilisateur donné qui ne fournirait pas de photo pour son compte, exemple :



Le programme doit être en POO et le code principal pour l'utiliser doit pouvoir ressembler à ça :

```
// Création de l'avatar
$avatar = new Avatar();
$avatar->setSize(6);
$avatar->defineColors(4);
```

```
//création du SVG en lui transmettant la taille et la matrice de
couleurs
$svg = new Svg($avatar->getSize(), $avatar->getRandom());
```

```
//appel du code SVG
$code = $svg->render();
```

Ici l'on créé un avatar qui fera 6 carrés sur 6, il contiendra 4 couleurs différentes. Puis on appelle notre générateur SVG auquel on transmet la taille et la matrice qui aura été générée par cet objet Avatar.

Il faut imaginer la matrice comme un tableau à 2 dimensions, où chaque case contient le code de couleur qu'elle doit afficher :

	#FF0000		#00FF00		#0000FF		#FFFF00		#FF0000		#00FF00	
	-----		-----		-----		-----		-----		-----	
	#FFFF00		#FF0000		#00FF00		#0000FF		#FFFF00		#FF0000	
	-----		-----		-----		-----		-----		-----	
	#0000FF		#FFFF00		#FF0000		#00FF00		#0000FF		#FFFF00	
	-----		-----		-----		-----		-----		-----	
	#00FF00		#0000FF		#FFFF00		#FF0000		#00FF00		#0000FF	
	-----		-----		-----		-----		-----		-----	
	#FFFF00		#FF0000		#00FF00		#0000FF		#FFFF00		#FF0000	
	-----		-----		-----		-----		-----		-----	
	#0000FF		#FFFF00		#FF0000		#00FF00		#0000FF		#FFFF00	
	-----		-----		-----		-----		-----		-----	

Création des classes

Vous aurez besoin de 3 classes pour réaliser cette exercice:

- la classe Utils contient une méthode **static** getRandomColor qui génère une couleur aléatoire
- la classe Avatar, contient les propriétés d'un avatar, c'est à dire sa taille, ses couleur et contient une méthode permettant, à partir de ces informations, de créer la matrice ci-dessus. Elle contient également une méthode 'defineColors' qui permet de définir le nombre de couleurs souhaitées et d'aller demander à Utils ces couleurs aléatoires.
- la classe Svg récupère taille et matrice lors de son instantiation et possède une méthode qui va générer le code SVG à partir de la matrice

L'héritage

Introduction

Un brochet peut nager, c'est l'une des propriétés de cet animal.

```
class Pike // Anglais pour « Brochet »
{
    public function swim(): void
    {
        echo 'Fsssschhh !';
    }
}
```

```
$pike = new Pike();
$pike->swim(); // Fsssschhh !
```

Le fait de pouvoir nager est-il une caractéristique spécifique au brochet ? Ce dernier appartient en réalité à la super-classe des poissons osseux, ou *osteichthyes*, que nous abrègerons ici par la simple désignation de poissons.

Les poissons nagent. Le brochet est un poisson. Donc le brochet nage.

Constatons-le :

```
class Zander // Anglais pour « Sandre »
{
    public function swim(): void
    {
        echo 'Fsssschhh !';
    }
}
```

```
$zander = new Zander();
$zander->swim(); // Fsssschhh !
```

Le sandre nage également.

Nous nous retrouvons avec deux classes partageant une méthode identique. Cela devrait vous soulever un sourcil. Souvenez-vous du principe **DRY**.

Héritage

Nous l'avons dit : nager n'est pas une action propre au brochet, ni au sandre, mais au poisson. En programmation, on dit que le brochet **hérite** du poisson. Le brochet hérite de tous les attributs et actions d'un poisson, comme posséder des branchies ou nager, en plus de ses attributs et actions propres, comme sa silhouette fusiforme ou ses capacités de chasse en embuscade.

Comment traduire cet **héritage** en PHP ?

```
class Fish
{
    public function swim(): void
    {
        echo 'Fsssschhh !';
    }
}
```

```
class Pike extends Fish
{
    public function ambush(): void
    {
        echo 'Bouh !'
    }
}
```

```
$pike = new Pike();
$pike->swim(); // Fsssschhh !
$pike->ambush(); // Bouh !
```

En tant que brochet, le brochet peut tendre des embuscades. En tant que poisson, le brochet peut nager. Voilà ce qu'exprime ce code. Et comme vous le voyez dans les dernières lignes, l'instance **\$pike** de la classe **Pike** est en effet capable d'appeler aussi bien la méthode **swim** que la méthode **ambush**.

Qu'avons-nous fait ?

À la première ligne de la classe **Pike**, juste après **class Pike**, nous avons ajouté le mot-clé **extends** suivi du nom de la classe **Fish**. Nous pourrions traduire cette ligne par « La classe **Pike** qui étend la classe **Fish** ». Ce verbe « étend » signifie tout simplement « hérite de », ou « est ».

« Le brochet est un poisson. »

« Brochet étend Poisson. »

« Brochet hérite de Poisson. »

Autant de façons d'exprimer l'héritage. En disant que la classe **Pike** étend la classe **Fish**, sous-entendu que le **Pike** est un **Fish**, on dit à PHP que le brochet doit disposer des attributs et actions d'un poisson puisqu'il est un poisson.

Cela nous amène à une notion très importante de la POO que vous devez retenir et bien comprendre : le **polymorphisme**. Une même idée abstraite, une même classe, peut prendre plusieurs formes concrètes.

Abstraction

Ajoutons un détail à notre exemple précédent :

```
abstract class Fish
{
    public function swim(): void
    {
        echo 'Fsssschhh !';
    }
}
```

```
class Pike extends Fish
{
    public function ambush(): void
    {
        echo 'Bouh !'
    }
}
```

```
$pike = new Pike();
$pike->swim(); // Fsssschhh !
$pike->ambush(); // Bouh !
```

La seule modification est à la toute première ligne. L'ajout du mot-clé **abstract** définit la classe **Fish** comme abstraite.

Nous vous invitons dans un chapitre précédent à penser à une voiture. Retentez l'expérience : pensez à un poisson.

Qu'avez-vous en tête ? Un poisson rouge ? Une perche ? Un gardon ? Une truite ? Le fait est que vous ne pouvez pas penser à la notion abstraite de « poisson », si vous essayez de visualiser un poisson vous verrez toujours une espèce concrète. Votre cerveau ne peut pas créer une instance de Poisson, il crée plutôt une instance de Truite.

En POO la logique est la même. Certaines classes ne sont vouées qu'à être des définitions abstraites, pensées pour être étendues mais pas pour être directement instanciées. Ici, la classe **Fish** ne pourra jamais être instanciée, on peut seulement en hériter. Toute tentative de **new Fish()** lèvera une erreur.

Il existe, à l'inverse du mot-clé **abstract**, une notion de **final**. Cependant, dans la mesure où un développeur débutant a toutes les chances de mal l'utiliser, nous préférons ne pas détailler ce concept ici. Sachez seulement que ça existe.

Substitution

Une sous-classe peut se **substituer** à la classe qu'elle étend. Cela signifie que non seulement le brochet hérite des propriétés et méthodes du poisson, mais qu'il peut se faire concrètement passer pour un poisson, il est un poisson. Et ça, PHP le comprend bien.

```
class Model
{
}
```

```
class Article extends Model
{
}
```

```
$article = new Article();
echo $article instanceof Article; // true
echo $article instanceof Model;  // true
```

Dans cet exemple, on demande à PHP si **\$article** est de type **Article**. Il répond tout naturellement que oui. Puis on lui demande si **\$article** est de type **Model** : toujours oui.

C'est un découlement logique du principe de **polymorphisme** : un modèle peut apparaître sous plusieurs formes : un article, un utilisateur, une catégorie, peu importe. Quelle que soit sa forme, un modèle reste un modèle.

Visibilité

Encore cette encapsulation ?

Oui, **la visibilité** a son importance en héritage. Une sous-classe hérite des propriétés et méthodes des classes qu'elle étend... à condition que lesdites propriétés et méthodes ne soient pas privées.

```
abstract class Post
{
    private int $id;
}
```

```
class Article()
{
    public function getId(): int
    {
        return $this->id;
    }
}
```

```

    }

    public function setId(int $id): void
    {
        $this->id = $id;
    }
}

```

```

$article = new Article();
$article->setId(12);

```

Ce code n'est pas fonctionnel, il lèvera une erreur. La visibilité privée est très stricte et ne fait pas d'exception dans le contexte d'héritage. La classe `Article` ne peut pas accéder à la propriété `id`.

Une solution serait de la passer en visibilité publique. Mais à ce moment, on perdrait les bénéfices de l'encapsulation puisque n'importe quel morceau de code y aurait accès.

Comment contourner ce problème ? Comment rendre cette propriété accessible de l'intérieur de la classe et des classes qui en héritent, sans pour autant la rendre accessible de l'extérieur ?

Il existe une troisième visibilité : la visibilité protégée.

```

abstract class Post
{
    protected int $id;
}

class Article()
{
    public function getId(): int
    {
        return $this->id;
    }

    public function setId(int $id): void
    {
        $this->id = $id;
    }
}

```

```

$article = new Article();
$article->setId(12);

```

La propriété `id` a été passée en `protected`. Ce code fonctionne.

Cet exemple demeure très étrange du point de vue du design. Ce n'est pas une très bonne façon de faire. En fait, il est difficile de trouver un exemple simple à

l'utilisation de `protected`, car son emploi n'est pas si courant et se justifie dans des exemples plus complexes que celui-ci. Nous ne voulons pas vous embrouiller.

Ce que vous pouvez retenir, c'est qu'il faut utiliser en priorité une combinaison de `private` et de `public`. La visibilité `protected` existe, elle remplit un certain rôle, mais n'en abusez pas si `private` peut suffire. La pratique et l'expérience vous apprendront à manier l'encapsulation avec sagesse.

Profondeur

Il n'y a techniquement pas de limite à la profondeur d'une chaîne d'héritage.

```
class Food
{
}

class Fruit extends Food
{
}

class Banana extends Fruit
{
}
```

La banane étend le fruit, qui étend la nourriture. C'est un héritage à trois niveaux, et ça fonctionne très bien.

Si on le souhaite, on peut même définir un héritage à dix niveaux, cent niveaux. C'est techniquement, théoriquement possible.

Mais... ne le faites pas.

N'abusez pas de la profondeur en héritage. Deux niveaux sont largement suffisants dans la grande majorité des cas. Trois niveaux parfois, allez. Si vous commencez à avoir besoin de quatre, cinq ou plus de niveaux, cela révèle très probablement un design mal pensé : remettez votre architecture en question.

Redéfinition

Souvent confondue avec la `surcharge`, qui est une mécanique qui n'existe pas en PHP mais qu'on retrouve par exemple en java ou en python, la **redéfinition** est une notion de POO liée à l'héritage.

Le principe est très simple.

```
abstract class Fish
{
    public function swim(): void
    {
        echo 'Fsssschhh !';
    }
}
```

```
class Pike
{
    public function swim(): void
    {
        echo 'Zap !';
    }
}
```

```
$pike = new Pike();
$pike->swim(); // Zap !
```

Les poissons nagent. En tant que poisson, le brochet nage.

MAIS. Admettons que le brochet ait une manière de nager qui lui est très propre : très nerveuse, rapide et en ligne droite sur de courtes distances. La plupart des poissons partagent une manière de nager assez similaire, mais le brochet a son petit truc à lui.

Dans ce cas, nous avons la possibilité de **redéfinir** la méthode `swim`. Nous écrasons ici la méthode `swim` normalement héritée de `Fish` pour en définir une nouvelle version plus adaptée aux comportements spécifiques de la sous-classe `Pike`.

C'est très simple. Pas si souvent utilisé, mais ça existe !

Résumé

À retenir :

- L'héritage est une façon de dire qu'une classe A est une extension d'une classe B, héritant par conséquent des propriétés et méthodes qui lui sont accessibles ;
- Puisque les classes `Article` et `User` héritent d'une unique classe `Model`, on dit alors que – au nom du principe de **polymorphisme** – `Article` et `User` se **substituent** à `Model` : PHP considère que `Article` est du type `Article` mais aussi du type `Model` ;
- La visibilité `protected` est un genre de `private` étendu qui rend une propriété ou méthode accessible de l'intérieur d'une classe ainsi que de l'intérieur de ses sous-classes ;

- Le mot-clé **abstract** empêche une classe d'être directement instanciée, la vouant à ne pouvoir être qu'étendue par une autre classe ;
- Il n'y a pas de limite de profondeur à une chaîne d'héritage, même s'il est très rare d'avoir une bonne raison de dépasser deux ou trois niveaux ;
- Une méthode définie dans une classe parent peut être redéfinie par ses sous-classes.

Les interfaces

Introduction

Souvenez-vous : dans notre **introduction** à la POO, nous avons évoqué la distinction entre l'**interface** et l'**implémentation**.

L'interface est le contrat abstrait passé entre deux modules, et l'implémentation est l'application pratique de ce contrat.

Un feu rouge vous impose de vous arrêter, c'est le contrat que vous passez en acceptant de respecter le Code de la Route. Si vous êtes en voiture, il faudra presser la pédale de frein qui activera des plaquettes qui ralentiront vos roues par frottement. En vélo, le mécanisme se situera au niveau des mains et activera des patins. Frein de voiture et frein de vélo, deux implémentations différentes pour remplir le même contrat et parvenir au même résultat final : l'arrêt.

Dans la vie quotidienne, rien d'autre que le contrat social ne vous empêche techniquement d'enfreindre les règles.

En PHP il existe une mécanique permettant de forcer une classe à respecter un contrat.

Interfaces

Les interfaces sont une mécanique existant dans quelques langages orientés objet : PHP mais aussi java et C#, entre autres.

```
interface ModelInterface
{
    public function getId(): int;

    public function setId(int $id): void;
}

class User implements ModelInterface
{
    private int $id;

    public function getId(): int
    {
        return $this->id;
    }
}
```

Le mot-clé **interface** suivi du nom de l'interface en *PascalCase*. Le nom finit conventionnellement par **Interface**, nous permettant de plus facilement les reconnaître dans le code.

Puis on définit la signature d'une ou plusieurs méthodes publiques. La signature d'une méthode ou d'une fonction, c'est, pour dire les choses simplement, sa première ligne : son nom, ses paramètres et types d'entrée, et son type de sortie.

Que dit ce code ? Que toute classe signant le contrat **ModelInterface** doit impérativement implémenter les méthodes **getId** et **setId**, en respectant précisément leurs signatures respectives. Si l'interface **ModelInterface** impose un **public function setId(int \$id): void;** et qu'on spécifie que la classe **User** est une implémentation de cette interface, alors cette classe **User** **doit** implémenter une méthode **setId** prenant un paramètre entier en entrée et ne renvoyant aucune valeur en sortie.

L'interface dit « Je veux que tout modèle ait obligatoirement ces méthodes. Je vous dis pas comment, je me fiche du détail, je veux juste qu'on puisse lire et modifier un ID numérique sur n'importe quel modèle. »

Le code de notre exemple n'est donc pas fonctionnel, il manque la méthode `setId`. Nous **devons** l'implémenter :

```
interface ModelInterface
{
    public function getId(): int;

    public function setId(int $id): void;
}

class User implements ModelInterface
{
    private int $id;

    public function getId(): int
    {
        return $this->id;
    }

    public function setId(int $id): void
    {
        $this->id = $id;
    }
}
```

Substitution

Comme pour l'héritage, le principe de substitution s'applique à l'implémentation d'une interface. Si une classe `ArticleController` implémente l'interface `ControllerInterface`, alors PHP considèrera qu'une instance de `ArticleController` est aussi bien de type `ArticleController` que de type `ControllerInterface` :

```
interface ControllerInterface
{
}

class ArticleController implements ControllerInterface
{
}

$controller = new ArticleController();
echo $controller instanceof ArticleController; // true
echo $controller instanceof ControllerInterface; // true
```

À quoi ça sert ?

Les interfaces apportent de la solidité au polymorphisme dans une application. En imposant à certaines classes d'implémenter certaines méthodes, on rend dans beaucoup de cas notre code beaucoup plus fiable et prévisible.

Beaucoup de cas ? Oui, principalement quand on aborde la généricité de notre code et qu'on développe des modules génériques et réutilisables.

Considérons cet exemple :

```
interface ModelInterface
{
    public function validate(): bool;
}

class User implements UserInterface
{
    private int $id;

    private string $email;

    private string $password;

    public function getId(): int
    {
        return $this->id;
    }

    public function setId(int $id): void
    {
        $this->id = $id;
    }

    public function getEmail(): string
    {
        return $this->email;
    }

    public function setEmail(string $email): void
    {
        $this->email = $email;
    }

    public function getPassword(): string
    {
        return $this->password;
    }

    public function setPassword(string $password): void
    {
        $this->password = $password;
    }
}
```

```

    public function validate(): bool
    {
        if (!filter_var($this->email, FILTER_VALIDATE_EMAIL))
        {
            return false;
        }

        if (strlen($this->password) < 8) {
            return false;
        }

        return true;
    }
}

```

En imposant à tout nouveau modèle d'implémenter une méthode `validate`, nous nous simplifions la vie à la réception des formulaires. Quel que soit le modèle en question, on sait qu'on peut lui faire confiance pour que cette méthode existe. De là, nous pouvons facilement automatiser la validation des formulaires, par exemple.

Est-ce que j'en ai besoin ?

Nous savons que les interfaces sont une notion avancée de la POO et qu'elles doivent vous sembler floues pour le moment. La seule théorie ne pourra jamais totalement vous faire comprendre quand les utiliser.

C'est par l'expérience que vous finirez par vous débloquer sur ce sujet : en voyant des développeurs expérimentés en créer et s'en servir, en les voyant dans le code de *frameworks* comme *Symfony*, en les utilisant vous-mêmes avant de chercher à en créer, etc.

Bref, pas de panique si à l'issue de ce chapitre vous ne comprenez toujours pas grand-chose aux interfaces : c'est normal. C'est une notion complexe qui demande de l'expérience. Ça viendra.

Résumé

À retenir :

- Une interface définit un contrat que doit respecter toute classe qui l'implémente ;
- Comme en héritage, les classes implémentant une interface se substituent à cette interface ;
- Les interfaces sont une notion conceptuellement difficile d'accès pour des développeurs débutants, pas de panique, vous les comprendrez mieux avec le temps.

Les traits

Introduction

Les traits sont une mécanique de polymorphisme qui est présente dans quelques langages, comme par exemple le java, le rust, et bien sûr le PHP.

Ils permettent de rendre les classes modulables.

```
trait Communications
{
    public function sayMessage(string $message): void
    {
        echo $message;
    }
}
```

```
class User
{
    use Communications;

    private int $id;

    public function getId(): int
    {
        return $this->id;
    }
```

```
    public function setId(int $id): void
    {
        $this->id = $id;
    }
}
```

```
$user = new User();
$user->sayMessage('Salut'); // Salut
```

Dès la première ligne de la classe, en spécifiant que **User** utilise le trait **Communications**, on rend alors toutes les méthodes de **Communications** accessibles à **User**.

Quelle différence avec l'héritage ?

Une classe ne se substitue pas à ses traits du point de vue du typage. Dans notre exemple, `User` n'est **pas** de type `Communications`.

`Communications` n'est qu'une simple collection de méthodes.

Ensuite, les traits répondent à un angle mort de l'héritage. En PHP, une sous-classe ne peut hériter que d'une seule classe. De là, que faire si on a besoin de fonctionnalités communes à plusieurs sous-classes d'un même héritage, mais pas sur toutes les sous-classes de ce même héritage ?

Partons du principe que les requêtes SQL liées à nos modèles sont gérées par des `ModelManager` associés. Pour un `Article`, un `ArticleManager`. De là, imaginons l'exemple suivant : certains managers permettent des requêtes SQL de recherche dans le texte. Cette fonctionnalité peut être utile par exemple pour des modèles comme les articles ou les utilisateurs, mais moins pertinente pour des catégories ou des adresses. Ainsi, puisque tous les modèles ne vont pas avoir besoin de recherche full-text, nous n'allons pas placer notre méthode `fullTextSearch` dans notre classe abstraite `ModelManager`. Il nous reste la solution de la placer dans chaque sous-classe de `ModelManager`, mais cette option contrevient au principe DRY.

C'est là qu'interviennent les traits.

```
trait Searchable
{
    public function fullTextSearch(string $table, array $fields, string $term): array | bool
    {
        // Some example way to get the PDO
        $pdo = $this->getPdo();

        // This is an example. In real life, of course, you would have to
        // be very careful about SQL injection here.
        $sql = "SELECT * FROM $table";
        for ($i = 0, $c = count($fields); $i < $c; ++$i) {
            if ($i > 0) {
                $sql .= ' OR';
            }

            $sql .= " WHERE $fields[$i] LIKE :term"
        }

        $query = $pdo->prepare($sql);
        $query->execute([
            ':term' => "%$term%"
        ]);
    }
}
```

```
        return $query->fetchAll();
    }
}
```

```
abstract class ModelManager
{
    // ...
}
```

```
class ArticleManager extends ModelManager
{
    use Searchable;

    // ...
}
```

```
class UserManager extends ModelManager
{
    use Searchable;

    // ...
}
```

```
class CategoryManager extends ModelManager
{
    // ...
}
```

Ainsi, certains de nos **ModelManager** bénéficient de la fonctionnalité générique de recherche full-text, mais nous épargnons cette méthode aux classes qui n'en ont pas besoin.

Est-ce que j'en ai besoin ?

En pratique, les traits ne sont pas tant utilisés que cela dans l'écosystème PHP et le monde professionnel. L'exemple que nous vous avons présenté n'est pas des plus simples, et pour cause : l'emploi des traits se légitime dans des situations très particulières. Il faut prendre garde à ne pas en abuser, en privilégiant notamment l'héritage quand celui-ci est plus pertinent.

Comme pour les interfaces, comprendre quand utiliser les traits et quand ne pas les utiliser viendra avec le temps et la pratique.

Résumé

À retenir :

- Les traits permettent d'ajouter des fonctionnalités à des classes, c'est une forme de polymorphisme plus souple que l'héritage mais moins puissante et moins solide ;
- Le principe de substitution ne s'applique pas aux traits.

Bot pokemon

Énoncé de l'exercice : Créer un jeu Pokémon avec un bot chasseur de Pokémon

Dans cet exercice, vous allez créer un petit jeu en PHP en utilisant les concepts de Programmation Orientée Objet (POO). Le but est de simuler un combat Pokémon où un bot tente de capturer un Pokémon.

Les spécifications :

La classe **Pokemon** :

- La classe **Pokemon** doit être **abstraite** et ne peut pas être instanciée.
- Elle doit inclure des propriétés comme :
 - **name** : le nom du Pokémon.
 - **max_life** : les points de vie maximum.
 - **life** : les points de vie actuels.
 - **level** : le niveau du Pokémon.
 - **type** : le type du Pokémon.
 - **strength** : la force d'attaque.

- La classe doit contenir des méthodes :
 - `__construct($name, $max_life, $life, $level, $type, $strength)` pour initialiser les propriétés du Pokémon.
 - `attack($pokemon)` pour infliger des dégâts à un autre Pokémon selon la formule : `strength * (rand(900, 1100) / 1000)`, arrondi à l'entier supérieur.
 - `take_damages($damages)` pour réduire les points de vie du Pokémon en fonction des dégâts reçus.
 - `heal($heal)` pour soigner le Pokémon d'un montant donné, sans dépasser ses points de vie maximum.
 - `is_dead()` pour vérifier si le Pokémon est KO.
 - La classe doit définir deux méthodes abstraites :
 - `level_up()` pour gérer la montée en niveau.
 - `say_hi()` pour afficher un message propre à chaque Pokémon.

Les Pokémon de départ :

Implémentez trois classes représentant les starters : `Carapuce`, `Salameche` et `Bulbizarre`.

1. `Carapuce` gagne 9 PV par niveau et a une force d'attaque de 2.
2. `Salameche` gagne 5 PV par niveau et a une force d'attaque de 4.
3. `Bulbizarre` gagne 7 PV par niveau et a une force d'attaque de 3.

Les pokémons auront un constructeur qui prendra en argument uniquement leur niveau. Si vous le souhaitez, vous pouvez ensuite utiliser le constructeur parent avec la propriété `parent::` (voir l'exemple 3 de la doc php <https://www.php.net/manual/fr/language.oop5.paamayim-nekudotayim.php>).

Interface `Usable` :

1. Créez une interface `Usable` que tous les objets utilisables devront implémenter.
2. Elle doit contenir une méthode `use($target)`.

Classe `Ball` :

Créez une classe abstraite `Ball` qui représente une Pokéball avec les propriétés suivantes :

1. `name` : le nom de la Pokéball.
2. `level` : le niveau de la Pokéball (10, 30, 50 et 1000).

Créez une classe pour chaque pokéball :

1. `Pokeball`
2. `Superball`
3. `Hyperball`
4. `Masterball`

1. Implémentez une méthode `try_catch($pokemon)` qui calcule si le Pokémon est capturé selon la formule suivante :
 - $((\$max_life - \$life) / \$max_life) * (1 + (\$lvl_pokeball - \$lvl_pokemon) / 25)$, arrondi au centième.
 - Tirez un nombre à virgule aléatoire entre 0 et 1, si le résultat de la formule précédent est supérieur ou égal au nombre aléatoire le pokémon est capturé, sinon la capture échoue.
2. Implémentez la méthode `use($target)` pour tenter de capturer un Pokémon.
3. Pour la Masterball le pokémon doit-être capturé à tous les coups.

Classe pour les soins :

1. Créez une classe `Heal` et dérivez-la pour créer :
 - `Potion` (récupère 20 PV).
 - `SuperPotion` (récupère 50 PV).
 - `HyperPotion` (récupère 200 PV).
 - `PotionMax` (restaure 100% des PV).
2. La classe `Heal` doit être un `Usable`.

Le bot :

Créez un bot qui utilise un Pokémon `Carapuce` (niveau 5) pour combattre un `Salameche` (niveau 8).

Vous disposez au départ des objets suivants :

- 3 Pokeball
- 2 Potion
- 1 SuperPotion

Le bot doit :

- Combattre le Pokémon adverse en alternant entre attaques et tentatives de capture.
- Utiliser les potions disponibles pour soigner son Pokémon si nécessaire.

Le combat s'arrête si :

- Le Pokémon adverse est capturé.
- Le Pokémon adverse est KO.
- Le Pokémon du bot est KO.